Quafu-Qcover: Explore combinatorial optimization problems on cloud-based quantum computers

Hong-Ze Xu(许宏泽), Wei-Feng Zhuang(庄伟峰), Zheng-An Wang(王正安), Kai-Xuan Huang(黄凯旋), Yun-Hao Shi(时运豪), Wei-Guo Ma(马卫国), Tian-Ming Li(李天铭), Chi-Tong Chen(陈驰通), Kai Xu(许凯), Yu-Long Feng(冯玉龙), Pei Liu(刘培), Mo Chen(陈墨), Shang-Shu Li(李尚书), Zhi-Peng Yang(杨智鹏), Chen Qian(钱辰), Yu-Xin Jin(靳羽欣), Yun-Heng Ma(马运恒), Xiao Xiao(肖骁), Peng Qian(钱鹏), Yanwu Gu(顾炎武), Xu-Dan Chai(柴绪丹), Ya-Nan Pu(普亚南), Yi-Peng Zhang(张翼鹏), Shi-Jie Wei(魏世杰), Jin-Feng Zeng(增进峰), Hang Li(李行), Gui-Lu Long(龙桂鲁), Yirong Jin(金贻荣), Haifeng Yu(于海峰), Heng Fan(范桁), Dong E. Liu(刘东), Meng-Jun Hu(胡孟军)
Citation: Chin. Phys. B, 2024, 33 (**5**): 050302. DOI: 10.1088/1674-1056/ad18ab
Journal homepage: http://cpb.iphy.ac.cn; http://iopscience.iop.org/cpb

What follows is a list of articles you may be interested in

Quantum generative adversarial networks based on a readout error mitigation method with fault tolerant mechanism

Run-Sheng Zhao(赵润盛), Hong-Yang Ma(马鸿洋), Tao Cheng(程涛), Shuang Wang(王爽), and Xing-Kui Fan(范兴奎)
Chin. Phys. B, 2024, 33 (**4**): 040304. DOI: 10.1088/1674-1056/ad02e7

Analysis of learnability of a novel hybrid quantum—classical convolutional neural network in image classification

Tao Cheng(程涛), Run-Sheng Zhao(赵润盛), Shuang Wang(王爽), Rui Wang(王睿), and Hong-Yang Ma(马鸿洋)
Chin. Phys. B, 2024, 33 (**4**): 040303. DOI: 10.1088/1674-1056/ad1926

Remote entangling gate between a quantum dot spin and a transmon qubit mediated by microwave photons

Xing-Yu Zhu(朱行宇), Le-Tian Zhu(朱乐天), Tao Tu(涂涛), and Chuan-Feng Li(李传锋)
Chin. Phys. B, 2024, 33 (**2**): 020315. DOI: 10.1088/1674-1056/ad1747

Variational quantum semi-supervised classifier based on label propagation

Yan-Yan Hou(侯艳艳), Jian Li(李剑), Xiu-Bo Chen(陈秀波), and Chong-Qiang Ye(叶崇强)
Chin. Phys. B, 2023, 32 (**7**): 070309. DOI: 10.1088/1674-1056/acb9fb

Energy shift and subharmonics induced by nonlinearity in a quantum dot system

Yuan Zhou(周圆), Gang Cao(曹刚), Hai-Ou Li(李海欧), and Guo-Ping Guo(郭国平)
Chin. Phys. B, 2023, 32 (**6**): 060303. DOI: 10.1088/1674-1056/acc521

COMPUTATIONAL PROGRAMS FOR PHYSICS

Quafu-Qcover: Explore combinatorial optimization problems on cloud-based quantum computers

Hong-Ze Xu(许宏泽)[1,†], Wei-Feng Zhuang(庄伟峰)[1,†], Zheng-An Wang(王正安)[1], Kai-Xuan Huang(黄凯旋)[1], Yun-Hao Shi(时运豪)[3,5,6], Wei-Guo Ma(马卫国)[3,5,6], Tian-Ming Li(李天铭)[3,5,6], Chi-Tong Chen(陈驰通)[3,5,6], Kai Xu(许凯)[3,1], Yu-Long Feng(冯玉龙)[1], Pei Liu(刘培)[1], Mo Chen(陈墨)[1], Shang-Shu Li(李尚书)[3,5,6], Zhi-Peng Yang(杨智鹏)[1], Chen Qian(钱辰)[1], Yu-Xin Jin(靳羽欣)[1], Yun-Heng Ma(马运恒)[1], Xiao Xiao(肖骁)[1], Peng Qian(钱鹏)[1], Yanwu Gu(顾炎武)[1], Xu-Dan Chai(柴绪丹)[1], Ya-Nan Pu(普亚南)[1], Yi-Peng Zhang(张翼鹏)[1], Shi-Jie Wei(魏世杰)[1], Jin-Feng Zeng(增进峰)[1], Hang Li(李行)[1], Gui-Lu Long(龙桂鲁)[2,1], Yirong Jin(金贻荣)[1], Haifeng Yu(于海峰)[1], Heng Fan(范桁)[3,1,5,6], Dong E. Liu(刘东)[2,1,4], and Meng-Jun Hu(胡孟军)[1,‡]

[1] *Beijing Academy of Quantum Information Sciences, Beijing* 100193, *China*
[2] *State Key Laboratory of Low Dimensional Quantum Physics, Department of Physics, Tsinghua University, Beijing* 100084, *China*
[3] *Institute of Physics, Chinese Academy of Sciences, Beijing* 100190, *China*
[4] *Frontier Science Center for Quantum Information, Beijing* 100184, *China*
[5] *School of Physical Sciences, University of Chinese Academy of Sciences, Beijing* 100190, *China*
[6] *CAS Center for Excellence in Topological Quantum Computation, UCAS, Beijing* 100190, *China*

(Received 8 November 2023; revised manuscript received 23 December 2023; accepted manuscript online 26 December 2023)

We introduce Quafu-Qcover, an open-source cloud-based software package developed for solving combinatorial optimization problems using quantum simulators and hardware backends. Quafu-Qcover provides a standardized and comprehensive workflow that utilizes the quantum approximate optimization algorithm (QAOA). It facilitates the automatic conversion of the original problem into a quadratic unconstrained binary optimization (QUBO) model and its corresponding Ising model, which can be subsequently transformed into a weight graph. The core of Qcover relies on a graph decomposition-based classical algorithm, which efficiently derives the optimal parameters for the shallow QAOA circuit. Quafu-Qcover incorporates a dedicated compiler capable of translating QAOA circuits into physical quantum circuits that can be executed on Quafu cloud quantum computers. Compared to a general-purpose compiler, our compiler demonstrates the ability to generate shorter circuit depths, while also exhibiting superior speed performance. Additionally, the Qcover compiler has the capability to dynamically create a library of qubits coupling substructures in real-time, utilizing the most recent calibration data from the superconducting quantum devices. This ensures that computational tasks can be assigned to connected physical qubits with the highest fidelity. The Quafu-Qcover allows us to retrieve quantum computing sampling results using a task ID at any time, enabling asynchronous processing. Moreover, it incorporates modules for results preprocessing and visualization, facilitating an intuitive display of solutions for combinatorial optimization problems. We hope that Quafu-Qcover can serve as an instructive illustration for how to explore application problems on the Quafu cloud quantum computers.

Keywords: quantum cloud platform, combinatorial optimization problems, quantum software

PACS: 03.67.Lx, 03.67.Ac, 42.50.Dv **DOI:** 10.1088/1674-1056/ad18ab

1. Introduction

Quantum computing represents a novel computing paradigm aimed at utilizing the principles of superposition and entanglement in quantum systems to accelerate computation. The fundamental unit for information processing in quantum computing is quantum bit, abbreviated as qubit. Different from classical bits, which can only exist in either the state 0 or 1, qubits can exist in a superposition of both states $|0\rangle$ and $|1\rangle$, enabling exponential representation capability of the state space. In order to achieve quantum computing, it is necessary to develop corresponding quantum algorithms. The key aspect of quantum algorithms lies in extracting the desired solution from the exponential state space through the design of suitable quantum operations on qubits. There are several quantum algorithms that have already been theoretically proven to have the ability to surpass their classical counterparts on certain issues, such as Shor, Grover, and HHL algorithms.[1,2] The Shor algorithm, for example, can factor a large integer in polynomial time, whereas the classical algorithm requires exponential time.[3,4] Similarly, the Grover algorithm can search for a specific item in an unsorted database with only $O(\sqrt{N})$ time complexity, which represents quadratic acceleration compared to the existing classical algorithm.[5] Furthermore, the HHL algorithm can solve some specific linear equation problems in

[†]These authors contributed equally to this work.
[‡]Corresponding author. E-mail: humj@baqis.ac.cn

polynomial time, whereas classical computers require exponential time to solve the same problems.[6]

In the past decade, with the rapid development of quantum technologies, various physical platforms have been constructed to demonstrate quantum computing tasks.[7–21] Among these platforms, the superconducting quantum platform has garnered significant attention due to its scalability and high fidelity of gate operations. In 2019, Google utilized its 53-qubit superconducting quantum processor "Sycamore" for the first time to achieve quantum supremacy in solving the random circuit sampling problem.[13] The record was updated in 2021 by the Pan group at USTC with a 66-qubit superconducting quantum processor named "Zuchongzhi II" to address the same problem.[14]

However, at present, all quantum computers lack fault-tolerance. To achieve fault-tolerant quantum computing, it would be necessary to have millions of physical qubits with high gate operation fidelity, which far exceeds current technological capabilities. Consequently, for the foreseeable future, we will be and remain in the era of "noisy intermediate scale quantum" (NISQ) for a long time. During the NISQ stage, quantum processors typically consist of tens to hundreds of physical qubits, yet they will be susceptible to noise and lack efficient error correction implementations.[22] Due to the limitations in qubits quality and gate operations, the depth of executable quantum circuits will be severely restricted. As a result, only shallow quantum circuits can be executed efficiently. The most urgent and challenging task during the NISQ era is to illustrate the quantum advantages in application scenarios.

There are some fields, such as molecular simulation, material design, and combinatorial optimization, that are considered suitable for exploration in the NISQ era.[23] Among these, combinatorial optimization has attracted a lot of attention and research due to its involvement in various industrial production scenarios, including logistics, finance, and telecommunications. Combinatorial optimization problems, known for being NP-hard, lack classical algorithms with polynomial complexity as the problem scale increases. Quantum algorithms are expected to outperform classical algorithms in finding optimal solutions. In 2014, Farhi *et al* introduced the quantum approximation optimization algorithm (QAOA) for solving combinatorial optimization problems.[24] QAOA is a variational algorithm, making it well-suited for implementation on NISQ processors. Its fundamental concept involves transforming optimization problems into the search for ground state solutions of the Ising Hamiltonian, which can be achieved using the variational method. To date, various variants of QAOA have been proposed, including recursive QAOA,[25] warm-start QAOA,[26] and reinforcement learning-assisted QAOA,[27] among others. While QAOA does have certain limitations,[28–31] QAOA-related algorithms have shown the potential to surpass classical algorithms in specific instances.[25,32,33]

In recent years, there has been rapid evolution in superconducting-based quantum computing cloud platforms, exemplified by IBM Quantum. To date, IBM Quantum has launched more than 20 quantum processors on the cloud, ranging from 5 qubits to a maximum of 127 qubits.[34–38] The cloud-based quantum computing service allows users to remotely access quantum computers and execute quantum programs, making quantum resources accessible to a broader audience. The IBM Quantum alone has already served millions of registered users, and billions of quantum circuit tasks have been executed by the quantum cloud platform. As more and more people use and explore quantum computing in the cloud, there is a significant need to present guiding examples of full-stack application software to users. On the one hand, it helps beginners explore quantum computing in specific application fields without a professional background in quantum computing. On the other hand, it provides potential high-level quantum developers with a reference example to develop their own cloud-based quantum software.

Therefore, in this work, we present Quafu-Qcover, an open-source quantum software designed to assist users in exploring combinatorial optimization problems using the QAOA algorithm. Quafu-Qcover enables direct access to quantum computers through the Quafu quantum computing cloud platform.[39] The fundamental workflow of Quafu-Qcover comprises four main stages. Firstly, combinatorial optimization problems can be effectively represented as a quadratic unconstrained binary optimization (QUBO) model, which can be equivalently transformed into the Ising Hamiltonian form through appropriate variable transformations. Secondly, the QAOA circuit is constructed based on the Ising Hamiltonian with unspecified parameters. The optimal parameters are found using the graph decomposition method, where each subgraph corresponds one-to-one with smaller QAOA circuits that can be executed on quantum hardware or simulators.[40] Thirdly, the QAOA circuit is compiled into an executable quantum circuit on the Quafu quantum computers through an efficient compiler. It should be noted that the determination of optimal parameters and circuit compilation can be realized in parallel. Finally, the compiled circuit is sent to Quafu quantum computers to execute sampling, while enabling users to monitor the task status at any time using the task ID. Upon completion of the sampling task, Quafu-Qcover returns the problem results in a visualization format along with the bit strings of the QUBO model.

It should be emphasized that to execute quantum algorithms on practical quantum computers, the quantum programs have to be converted into hardware-executable quantum gate sequences or pulse sequences through a quantum

compiler. Quantum compilation is the crucial link that connects quantum programs and quantum hardware. In the current NISQ era, the primary objective of quantum compilation is to optimize quantum circuits by reducing the circuit depth and the number of two-qubit gates, while considering the topology and features of the underlying hardware structure. Although it has been demonstrated that finding the optimal quantum circuit is typically an NP-hard problem,[41–43] in practice, our focus is on searching for sub-optimal quantum circuits within a feasible timeframe. For certain circuits with special structures, such as the QAOA circuit, particular compilation strategies can outperform general-purpose compilers.[44–48] Inspired by previous works and taking into consideration the topology and characteristics of the Quafu quantum processor, we develop a dedicated compiler for Quafu-Qcover. This compiler demonstrates superior performance in contrast to general-purpose compilers such as Qiskit.

The remaining part of this paper is organized as follows. In Section 2, a comprehensive overview of the fundamental background of the QUBO model and the QAOA is provided. In Section 3, the Quafu-Qcover software framework is introduced and the detailed functionalities of each module are discussed. Following that, Section 4 demonstrates the practical application of Quafu-Qcover by presenting a specific combinatorial optimization problem and outlining the step-by-step process. Lastly, Section 5 offers a comprehensive summary of the entire paper and engages in a discussion concerning potential avenues for future research.

2. QUBO and QAOA basics

Combinatorial optimization problems play a significant role in various applications,[49] including maximum independent set (MIS) problems, maximum cut (MaxCut) problems, graph coloring problems, task allocation problems, and others. These diverse combinatorial optimization problems can be effectively modeled using a universal framework known as the QUBO form. The QUBO model has gained significant attention in the field of quantum annealing and is a key component in experiments conducted with quantum annealers developed by D-Wave Systems.[49,50] Additionally, it serves as the foundational framework for the QAOA, which operates on gate-based quantum computers. The objective function of the QUBO model can be expressed as follows:

$$f(x)=\sum_{i,j}Q_{ij}x_ix_j=x^{\mathrm{T}}Qx,\quad x_i\in\{0,1\},\tag{1}$$

where $x\in\{0,1\}^n$ represents a binary variable vector, and $Q\in\mathbb{R}^{n\times n}$ denotes a real square matrix. In combinatorial optimization problems, the goal is to optimize the objective function and identify the binary vector x^* that minimizes $f(x)$. That is

$$x^*=\min_{x\in\{0,1\}^n}f(x).\tag{2}$$

By defining $s_i=2x_i-1$, the QUBO model is inherently connected to the classical Ising model

$$H=\sum_{i,j}J_{ij}s_is_j+\sum_i h_is_i,\quad s_i\in\{-1,+1\}.\tag{3}$$

The problem of finding the optimal solution x^* in Eq. (2) is equivalent to determining the ground state of the Ising model, which means identifying the configuration of binary labels $\{-1,+1\}$ that minimizes the energy of the classical spin Hamiltonian.

Based on the above observations, it is natural to establish a mapping between combinatorial optimization problems and the quantum Ising model. The quantum Ising Hamiltonian is represented as follows:

$$H_{\mathrm{C}}=\sum_{i,j}J_{ij}Z_iZ_j+\sum_i h_iZ_i,\tag{4}$$

where Z_i represents Pauli-Z operators with eigenvalues $z_i=\pm1$, J_{ij} denotes the interactions between two spins, and h_i represents the biases associated with each individual spin. In the computational basis, the vectors $|z\rangle$ are represented by $z\equiv z_1z_2\cdots z_n$, and the Hamiltonian H_{C} is diagonal. Consequently, the objective function is defined as $C(z)=\langle z|H_{\mathrm{C}}|z\rangle$. The optimal solution to the combinatorial optimization problem requires identifying the bitstrings z that minimize or maximize the value of $C(z)$. However, the vast majority of optimization problems are recognized as NP-hard, implying that finding the optimal solution becomes considerably more challenging with an increase in problem size. Therefore, obtaining the exact optimal solution for such problems becomes almost impractical.

To address this challenge, Farhi and others proposed QAOA,[24] which is a variational quantum algorithm suitable for current NISQ processors. The QAOA typically introduces the following mixer Hamiltonian:

$$H_{\mathrm{B}}=\sum_{i=1}^{N}X_i,\tag{5}$$

where X_i are the Pauli-X operators. The QAOA aims to provide sub-optimal solutions for the objective Hamiltonian H_{C} by iteratively applying the objective Hamiltonian H_{C} and the mixer Hamiltonian H_{B} with optimized parameters. The initial state of the QAOA circuit is commonly prepared as a uniform superposition state of all n qubits. This state can be represented as follows:

$$|S\rangle=|+\rangle^{\otimes n}=\frac{1}{\sqrt{2^n}}\sum_z|z\rangle,\tag{6}$$

which is the eigenstate of $H_{\rm B}$. Subsequently, the unitary operators $U_{\rm C}(\gamma) = {\rm e}^{-{\rm i}\gamma H_{\rm C}}$ and $U_{\rm B}(\beta) = {\rm e}^{-{\rm i}\beta H_{\rm B}}$ are applied sequentially to the initial state in an alternating manner, resulting in the generation of a variational state denoted as

$$|\boldsymbol{\gamma},\boldsymbol{\beta}\rangle = U_{\rm B}(\beta_p)U_{\rm C}(\gamma_p)\cdots U_{\rm B}(\beta_1)U_{\rm C}(\gamma_1)|+\rangle^{\otimes n}. \tag{7}$$

This variational state is parameterized by $2p$ real variational parameters γ_i and β_i $(i = 1,2,\ldots,p)$, which control the angles of rotation associated with the objective Hamiltonian $H_{\rm C}$ and the mixer Hamiltonian $H_{\rm B}$, respectively. The expectation value of the objective Hamiltonian $H_{\rm C}$ in this variational state is given by

$$E_p(\boldsymbol{\gamma},\boldsymbol{\beta}) = \langle\boldsymbol{\gamma},\boldsymbol{\beta}|H_{\rm C}|\boldsymbol{\gamma},\boldsymbol{\beta}\rangle. \tag{8}$$

To initialize the parameters $\boldsymbol{\gamma}$ and $\boldsymbol{\beta}$, we can use either some random guesses or employ warm starting methods.[26,33] The latter provides strategies for initializing the parameters in a way that improves the convergence and performance of the QAOA algorithm. Next, classical optimizers are used to find the optimal parameters $\{\boldsymbol{\gamma}_{\rm opt},\boldsymbol{\beta}_{\rm opt}\}$ that minimize or maximize E_p. Based on the optimized parameters and Eq. (7), the corresponding quantum circuit can be prepared to obtain the best approximation solution state $|\boldsymbol{\gamma}_{\rm opt},\boldsymbol{\beta}_{\rm opt}\rangle$ using a quantum computer. By measuring in the computational basis, a bitstring z is obtained, and the value of $C(z)$ is evaluated. Repeating the measurements with the same optimized quantum circuit, the bitstring z^* that maximizes or minimizes $C(z)$ corresponds to an approximate optimal solution for the combinatorial optimization problem.

3. Software architecture

We now introduce the software architecture of Quafu-Qcover. The Quafu-Qcover is primarily composed of the following modules: user layer, core module, compiler module, quantum simulator backends, quantum hardware backends, and results preprocessing module, as shown in Fig. 1.

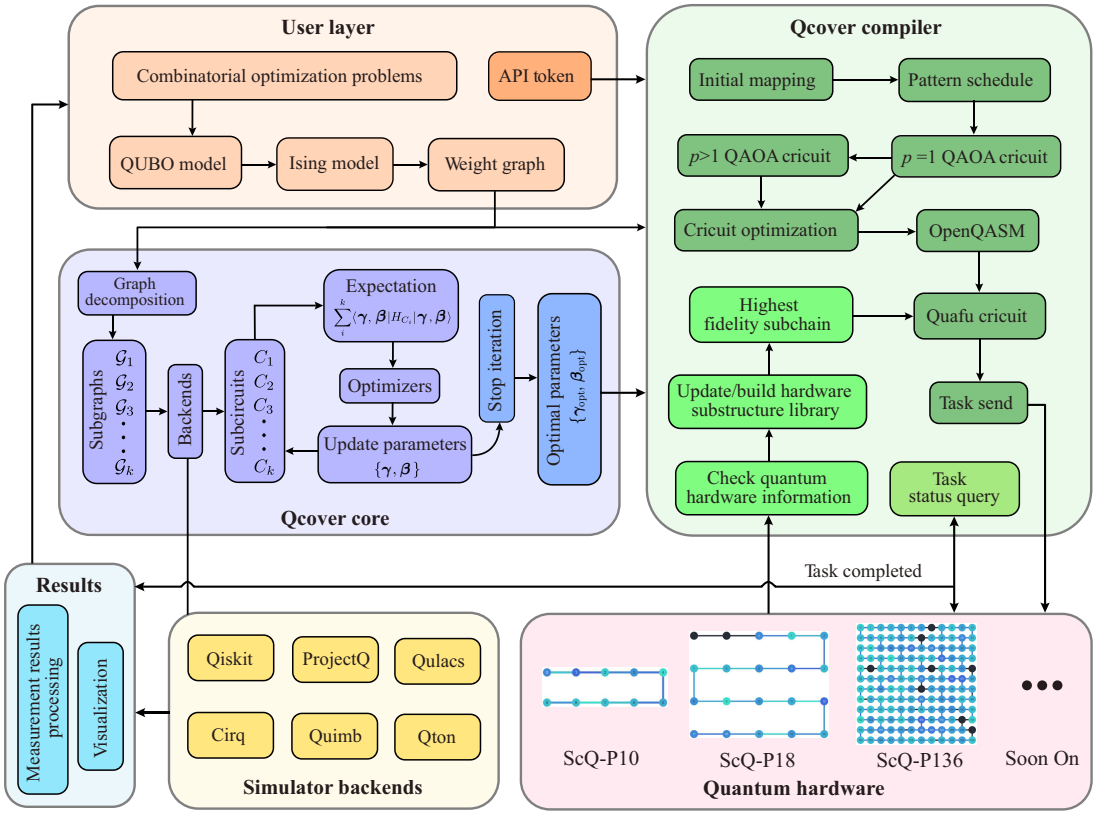

Fig. 1. The Quafu-Qcover software framework. At the user level module, users are required to input the combinatorial optimization problem to be solved and provide parameters such as the API token of Quafu. The Qcover core module is an essential component that utilizes the graph decomposition algorithm to determine optimal parameters for the QAOA circuit. The Qcover compiler module is used to compile the QAOA circuit into a physical quantum circuit that can be executed on Quafu hardware. The simulator backends consists of various open-source quantum programming packages. The quantum hardware encompasses the superconducting quantum chips currently accessible on the Quafu quantum computing cloud platform. The results module processes the sampling results obtained from Quafu and presents them to the users in a visual format.

3.1. User layer

The user layer serves as the interaction module between the users and the software. For a given combination optimization problem, we first need to model it in a QUBO form, and then transform the QUBO into the corresponding Ising Hamiltonian form. The Ising form itself cannot be directly processed, so it must be mapped to a weight graph $G(V,E)$ as the input for the next layer. The spins Z_i in the Ising Hamil-

tonian correspond to the vertices V_i in graph G, with h_i being the weights of the vertices. The Z_iZ_j terms correspond to the edges E_{ij} in G, with J_{ij} representing the weights of the edges. As for users, they only need to know how to model optimization problems in QUBO form. All necessary quantum components are already integrated into the software, allowing users to utilize them without needing a background in quantum theory. In addition, the *Qcover.applications* module of Quafu-Qcover contains pre-written examples of typical optimization problems that can be transformed into corresponding weighted graphs for computational purposes. To access quantum computers in the Quafu cloud, users need to provide their own API token, which can be easily obtained on the Quafu website.[39]

3.2. Core module

The core module of Quafu-Qcover focuses on finding the optimal parameters for the QAOA circuit using a graph decomposition algorithm. The fundamental idea involves using graph decomposition and graph-to-circuit mapping methods to divide a large-scale QAOA instance into multiple smaller independent instances, which can be processed in parallel using quantum simulators or hardware.[40] For most optimization problems, our algorithm exhibits a linear scaling of the running time with respect to the number of qubits. Most of the widely-used quantum simulators are compatible with Quafu-Qcover. We have integrated several popular simulator software packages as our backends, including Pyquafu,[39] Qiskit,[51] ProjectQ,[52] Qulacs,[53] $t|ket\rangle$,[54] and Quimb.[55] Additionally, we have also developed our own simulator called Qton,[56] which is also included in the software suite.

Quafu-Qcover provides several classic optimizers for updating parameters in QAOA circuits. These optimizers consist of gradient descent-based algorithms and other optimization algorithms such as COBYLA, L-BFGS-B, SHGO, SLSQP, and SPSA. Furthermore, some heuristic algorithms suitable for multi-layer QAOA circuits are also included, for example, Fourier and Interp algorithms.[57]

The core module follows a basic workflow as outlined below. Initially, the input weight graph $\mathcal{G}$ is decomposed into a series of small weighted subgraphs $\mathcal{G}_1, \mathcal{G}_2, \ldots, \mathcal{G}_k$ using the graph decomposition algorithm. Subsequently, each small weighted subgraph $\mathcal{G}_i$ is mapped to a corresponding QAOA subcircuit C_i based on the selected software backend framework. The subcircuits C_i correspond to different Ising Hamiltonians H_{C_i}, each containing unique terms from H_C without any overlap. To evaluate these subcircuits, we use selected simulator or hardware to calculate their expected values independently and parallelly. Then, the expectation value E_p of H_C can be determined as

$$E_p(\boldsymbol{\gamma},\boldsymbol{\beta}) = \sum_i^k \langle \boldsymbol{\gamma},\boldsymbol{\beta}|H_{C_i}|\boldsymbol{\gamma},\boldsymbol{\beta}\rangle. \tag{9}$$

Finally, a selected classical optimizer is used to optimize the parameters. The iteration stops and the optimal parameters are obtained when the expectation value reaches the desired accuracy or a predetermined threshold.

3.3. Compiler module

The introduction of the Qcover compiler marks another critical component of the Quafu-Qcover framework. This specialized compiler is purpose-built for creating QAOA circuits, enabling it to transform the weight graph of any given problem into an executable QAOA circuit suitable for quantum processors. Subsequently, this executable circuit is sent to the quantum hardware backend for computation. Next, we introduce the Qcover compiler.

To obtain the variational wave function, the quantum circuits can be constructed by integrating the unitary evolution operators $U_B(\beta_i)$ and $U_C(\gamma_i)$. Given that the terms in H_B are composed of commuting single-qubit operators, $U_B(\beta)$ is represented as the product of each term, as demonstrated below:

$$U_B(\gamma) = e^{-i\beta H_B} = \prod_l e^{-i\beta X_l}, \tag{10}$$

where each operator $e^{-i\beta X_l}$ corresponds to a single qubit $RX(2\beta)$ rotation gate. Regarding $U_C(\gamma)$, because all terms in H_C are diagonal in the computational basis, they commute with each other, leading to the expression of $U_C(\gamma)$ as

$$U_C(\gamma) = e^{-i\gamma H_C} = \prod_{lm} e^{-i\gamma J_{lm} Z_l Z_m} \prod_l e^{-i\gamma h_l Z_l}, \tag{11}$$

where the individual operators $e^{-i\gamma J_{lm} Z_l Z_m}$ and $e^{-i\gamma h_l Z_l}$ correspond to the two-qubits $RZZ(2\gamma J_{lm})$ gate and the single-qubit $RZ(2\gamma h_l)$ rotation gate, respectively.

As previously mentioned, when the Hamiltonian consists only of Pauli-Z operators, the terms commute with each other. This enables the unitary operator U_C to be precisely expressed in the form of the product in Eq. (11). However, when the Hamiltonian includes arbitrary Pauli operators, we can use the Trotter theorem to approximate the unitary operator U_C.[58,59] This paper focuses on the objective Hamiltonian consisting exclusively of Pauli-Z operators, which applies to most combinatorial optimization problems.

The original QAOA circuit needs to be compiled into a physical quantum circuit for execution on a real quantum processor. Several compilation methods have been developed for QAOA circuits. In this study, we primarily focus on a structured method introduced in Ref. [48]. This method shows that when executing the full connection weight graph on a one-dimensional quantum processor with nearest-neighbor coupling, the resulting QAOA circuit exhibits a structured form. Given a mapping of N logical qubits (q_i) onto a one-dimensional quantum processor with N physical qubits (p_i), the QAOA circuit is constructed by interleaving RZZ gates and SWAP gates as follows ($i = 0, 1, 2, \ldots$):[48]

(1) execute RZZ gate between all p_{2i} and p_{2i+1} physical qubits;

(2) execute RZZ gate between all p_{2i+1} and p_{2i+2} physical qubits;

(3) execute SWAP gate between all p_{2i+1} and p_{2i+2} physical qubits;

(4) execute SWAP gate between all p_{2i} and p_{2i+1} physical qubits.

To complete a graph with N nodes, the four steps mentioned above need to be repeated $\lceil N/2 \rceil$ times. Specifically, if N is even, $2N-2$ cycles are executed; if N is odd, $2N-1$ cycles are executed. The template circuit can be used for any incomplete graph by inserting the required RZZ gates and SWAP gates. It should be noted that different initial mappings may impact the circuit depth of the incomplete graph, but they do not affect the circuit depth of the complete graph. Nonetheless, the compilation approach is effective for any weighted graph, as the fixed nature of the compilation pattern enables the prediction of the position of the inserted RZZ gate. Once the initial qubit mapping is determined, the cycle of each RZZ gate in the circuit can be specified, facilitating the identification of the optimal mapping that minimizes the overall quantum circuit depth.

While it is feasible to calculate the circuit depth for a given initial mapping, obtaining the optimal initial mapping involves exploring a large number of possibilities, denoted by $N!$. As the graph size increases, it becomes impractical to explore all potential initial mappings. To improve the possibility of getting the optimal initial mapping, we use a heuristic search algorithm based on the approach outlined in Ref. [48]. This algorithm is based on two key observations: (1) When we map a new logical qubit q_i to a physical qubit p_j, we can use this pattern to determine the last cycle that q_i is executed among all other currently mapped qubits. (2) The vertices with higher connectivity in the weight graph have a more significant influence on the resulting circuit depth. Next, we introduce the heuristic search algorithm for finding the optimal initial mapping. In this algorithm, each node in the search tree represents a mapping of a logical qubit to a physical qubit. The algorithm flow is as follows:

Step 1 Arrange the vertices of the input problem graph in descending order according to their degrees and store them in the list Sq. The vertices with higher degrees are mapped first.

Step 2 Level 0 of the search tree comprises a single root node, initialized as empty, indicating the absence of any mappings. Following this, we proceed to generate child nodes for the root node by mapping the highest degree logical qubit $q = Sq[1]$ to each unmapped physical qubit. This action results in level 1 comprising N nodes, each associated with a cost of zero. Additionally, we add the mapped logical qubit $q = Sq[1]$ into the initially empty list Mq for future reference.

Step 3 The child nodes of level 2 are generated from the parent nodes of level 1. We map the second-highest degree logical qubit $q = Sq[2]$ to each unmapped physical qubit p, leading to level 2 containing $N(N-1)$ nodes. The cost of each node is determined as follows:

$$\mathrm{cost}_{q\to p} = \max_{m\in M_q \wedge m\leftrightarrow q} (\mathrm{ExeR}(q,m)), \tag{12}$$

where $m \leftrightarrow q$ indicates that the mapped logical qubit m and the current logical qubit q are connected in the input weight graph $\mathcal{G}$, and $\mathrm{ExeR}(q,m)$ represents the cycle at which the $\mathrm{RZZ}(q,m)$ gate is scheduled in the pattern. In order to avoid the excessive breadth of the search tree, we impose a truncation on each level. It is important to note that although node A costs less than node B under the current mapping, it does not guarantee that the node generated by node A will cost less than the node generated by node B when adding subsequent mapping. Hence, truncation in this case is not determined by the cost of the nodes. Rather, for nodes with equal cost, we keep $B_{\max}$ nodes and expand them. Since the cost corresponds to the cycle of the RZZ gate in the circuit, there can be a maximum of $2N$ distinct costs. As a result, the maximum breadth of the search tree is $2NB_{\max}$. Here, the maximum cycle is not expanded, and the constant term is disregarded. This approach allows us to efficiently manage the breadth of the search tree. We set $B_{\max} = 5$ unless otherwise specified. A larger value of $B_{\max}$ may yield a more optimal initial mapping, but it also leads to an increase in the search time. Upon completion of this step, we record the cost associated with the expanded node and include the mapped logical qubit $q = Sq[2]$ in the list Mq.

Step 4 Proceeding to step 3, we generate subsequent levels of the search tree. For instance, the child nodes of level 3 are generated from the nodes of level 2. This implies that we map the third-highest degree logical qubit $q = Sq[3]$ to each unmapped physical qubit p. Without truncating the search tree breadth, level 3 will contain $N(N-1)(N-2)$ nodes. The algorithm ends when all logical qubits are successfully mapped. Ultimately, we obtain a search tree with a maximum breadth of $2NB_{\max}$ and a depth of N. Each path in the search tree represents a distinct initial mapping. By considering the associated costs, we can identify the mapping that minimizes the overall circuit depth, serving as the optimal initial mapping.

The structured compilation method described above is applicable to the QAOA circuit with $p = 1$. According to Eqs. (10) and (11), we generalize this method to any integer p. When compiling the QAOA circuit for $p > 1$, the odd layers in the circuit have the same structure as the $p = 1$ circuit, while the even layers are reversed from the $p = 1$ circuit. An example is illustrated in Fig. 2(g). It is important to emphasize that the optimization parameters of each layer are different and require modification.

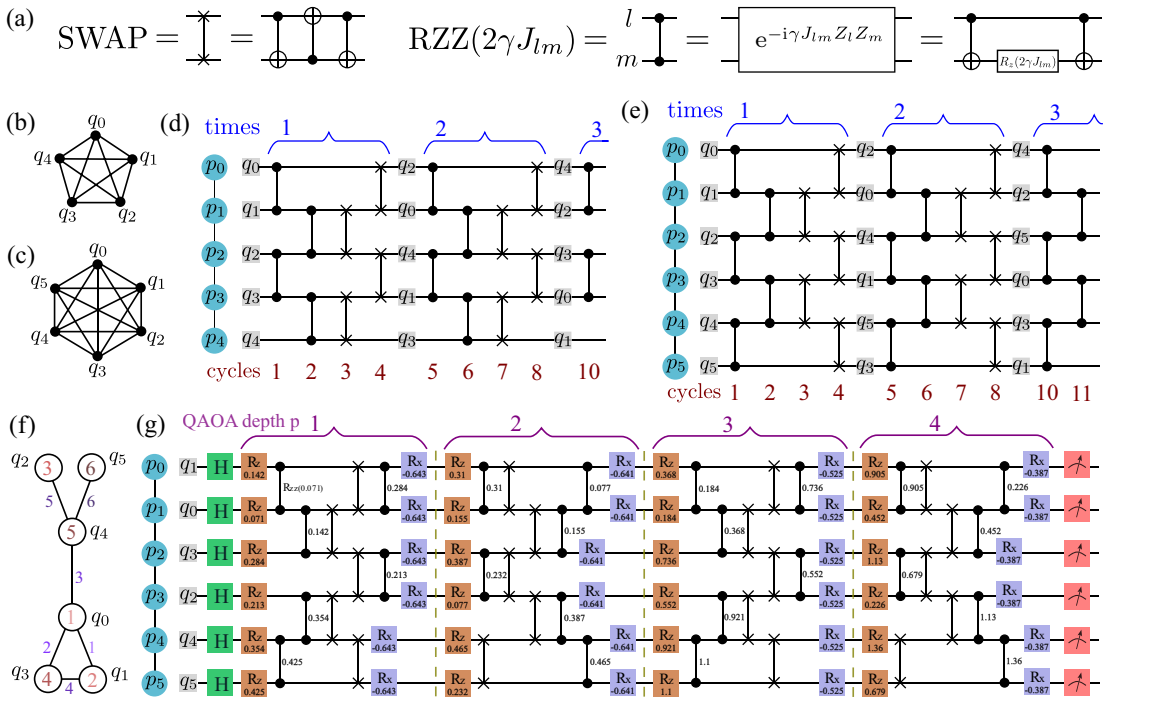

Fig. 2. (a) Decomposition rules for SWAP gates and RZZ gates. (b) and (c) are fully connected weight graphs with 5 nodes and 6 nodes, respectively. (d) and (e) correspond to the compilation templates of fully connected weight graphs with 5 nodes and 6 nodes on a one-dimensional chip, respectively. (f) A specific example of a weight graph for the Ising model, with values labeled on the nodes and edges indicating the weights. (g) The weight graph illustrated in (f) is compiled into a quantum circuit that conforms the one-dimensional qubits coupling structure using the Qcover compiler.

The quantum circuit obtained above contains RZZ gates and SWAP gates, which cannot be directly executed on superconducting hardware. Therefore, we need to decompose them into CNOT gates and RZ gates according to the rules shown in Fig. 2(a). Then, we further optimize the circuit by canceling adjacent CNOT gates with identical control and target qubits and aligning all gates to the left for earlier execution. This optimization results in fewer gates and a shallower circuit depth. The optimized circuit is then transformed into OpenQASM format. With these steps, we have successfully completed compiling any QAOA circuit on a one-dimensional superconducting chip. This method is also applicable to chips with arbitrary structures. We just need to identify the one-dimensional substructure chain of the chip and remap the qubits onto this subchain. Furthermore, our compiler can be transplanted to other qubit systems, such as trapped ion systems, by decomposing the RZZ gates and SWAP gates into the basic gate set supported by the specific quantum hardware.

In addition, as we are currently in the NISQ era with limited qubit fidelity, it is crucial to consider the fidelity information of qubits in quantum chips when executing quantum programs. The Qcover compiler includes the quantum hardware subchain library building module, which uses a heuristic algorithm to identify high overall fidelity subchains in chips with arbitrary coupling structures. In this context, "overall fidelity" refers to the product of the fidelities of all two qubits within the subchain. Due to the influence of the external environment on the fidelity of qubits, frequent calibration of qubits is necessary. The Qcover compiler module can dynamically update the subchain library based on real-time quantum hardware information. The subchain library is structured as a dictionary-type, with each entry containing a key-value pair. The key represents the number of qubits in the subchain, and the value represents the physical qubit coupling structure for each subchain. Please note that a single key in the dictionary may correspond to multiple subchains. The subchains are sorted in descending order based on their overall fidelity. When using the *pyquafu* to create a circuit, the subchain with the highest overall fidelity is selected from the library based on the required number of qubits. Subsequently, the logical qubits are sequentially mapped to the physical qubits of the selected subchain. Then, we use the *from_openqasm*() function in the *pyquafu* framework to transform OpenQASM code into *pyquafu* class circuits. Ultimately, we send the task to the Quafu quantum cloud platform for computation using the *send*() function.

3.4. Compiler performance benchmark

The effectiveness of a compiler can be evaluated using various performance indicators, such as the compilation time, circuit depth, and gate count of the resulting physical quantum circuit. These metrics are essential in assessing the efficiency and quality of the compilation process. To compare the performance of the Qiskit compiler and our Qcover compiler, we conducted experiments on a personal computer, evaluating their performance across different parameters such as the weight graph edge density $D = \text{edges}/C_N^2$, the number of nodes N, and the QAOA circuit depth p.

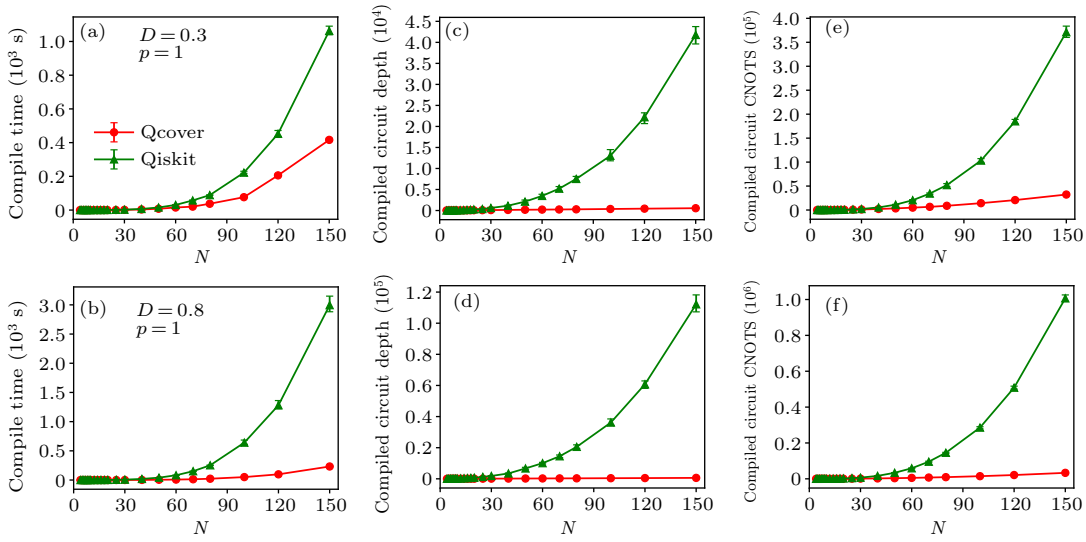

Fig. 3. Comparison of performance between the Qcover compiler and the Qiskit compiler. (a)–(b), (c)–(d), and (e)–(f) show the variation of compilation time, compiled circuit depth, and the number of compiled circuit CNOT gates with the number of weight graph nodes, respectively. The edge densities of the weight graphs in the upper and lower panels are 0.3 and 0.8, respectively. The Qiskit compiler selects the highest compilation optimization level. The green triangles and red dots represent the average results obtained by compiling 20 randomly generated weight graphs using the Qiskit compiler and the Qcover compiler, respectively.

In Fig. 3, we compare the results of compiling QAOA circuits with a depth of $p = 1$ using the Qiskit compiler and our Qcover compiler. The upper panel is for a random weight graph with an edge density of $D = 0.3$, while the lower panel displays a weight graph with a higher edge density of $D = 0.8$. The left, middle, and right panels show the compilation time, compiled circuit depth, and CNOT gate count as a function of the number of nodes, respectively. We randomly generated 20 weight graphs for a given number of nodes N and edge density D. Next, we compiled these weight graphs using the two compilers separately and averaged the results to provide a more comprehensive evaluation. In Figs. 3(a) and 3(b), it is evident that the Qcover compiler outperforms the Qiskit compiler by a significant margin. The compilation time of Qiskit is several to dozens of times longer than that of Qcover. In Figs. 3(c) and 3(d), the depth of the Qiskit compiled circuit grows exponentially as the number of nodes increases, while the depth of our Qcover compiled circuit increases linearly. In Figs. 3(e) and 3(f), we show the disparity in CNOT gate counts between the compiled circuits of the two compilers. The Qcover compiled circuit requires substantially fewer CNOT gates than the Qiskit compiled circuit. As the number of nodes increases, the CNOT gate count in the Qiskit compiled circuit grows exponentially, while in our Qcover compiled circuit it grows polynomially.

In Fig. 4, we present the performance of the two compilers under other various parameters. The upper panel displays how each performance metric varies with the edge density of the weight graph at $N = 100$ and $p = 1$. We observe that Qcover outperforms Qiskit for any edge density, with the improvement in Qcover performance becoming more noticeable as the edge density increases. As the edge density of the weight graph increases, the compilation time of Qiskit exhibits nearly linear growth, except for very small values of D. In contrast, the compilation time of Qcover demonstrates a decreasing trend. The reduced time required to find the optimal mapping in Qcover is due to the use of the template compilation pattern, which becomes more effective as the edge density approaches $D = 1$. The compiled circuit depth and the count of CNOT gates for Qiskit show a linear growth as the edge density of the weight graph increases. However, for Qcover, these quantities initially increase rapidly before tending to saturate as the edge density increases. The lower panel in Fig. 4 shows how the performance of the Qcover and Qiskit compilers varies with the QAOA depth p, for the weight graph with parameters $N = 100$ and $D = 0.8$.

It should be noted that these results are based on a single random weight graph. For different values of p, the compilation performance of Qcover is superior to that of Qiskit, and the performance improvement is nearly independent of p. The primary reason for this observation is that as the QAOA depth p increases, both Qiskit and Qcover demonstrate linear growth in compiled circuit depth and the count of CNOT gates. Furthermore, it is important to note that for each layer of the QAOA circuit, we can parallelly carry out optimal parameter configuration and circuit optimization. This implies that the compilation time will primarily depend on the QAOA circuit with $p = 1$. However, in this study, the compilation process was carried out sequentially. In future versions of Quafu-Qcover, it will support parallelization and parametric QAOA circuit compilation. In conclusion, the significantly enhanced performance of the Qcover compiler compared to Qiskit has been confirmed through our comprehensive evaluation of various parameters and performance metrics. This further underscores the importance of designing dedicated compilers for specific classes of application algorithms.

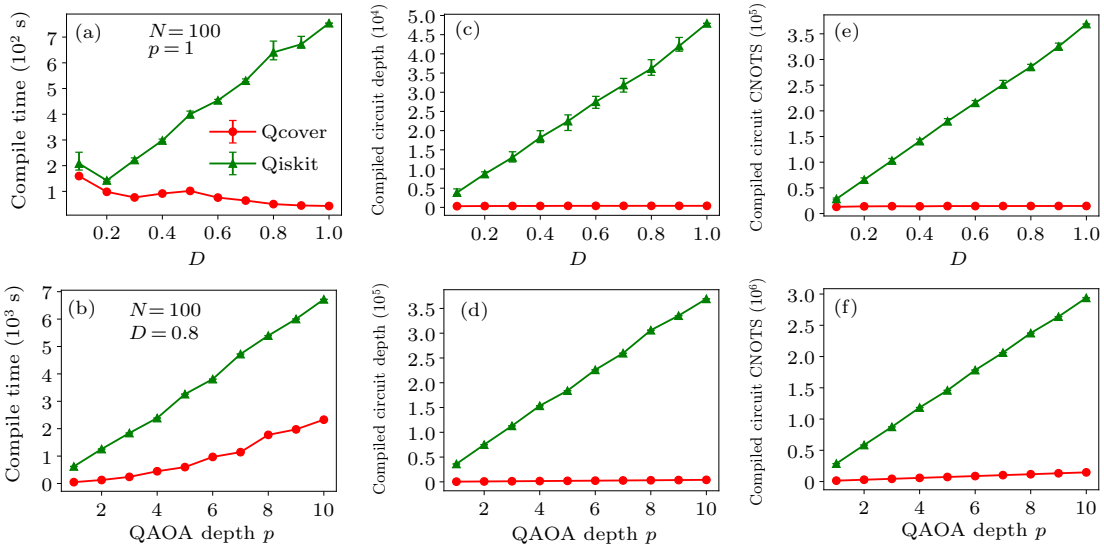

Fig. 4. The upper panel shows how compiler performance varies with the density of weight graph edges for a fixed weight graph with $N = 100$ nodes and QAOA circuit depth $p = 1$. The Qcover compiler significantly outperforms Qiskit by one to two orders of magnitude, with the performance improvement becoming more substantial as the edge density increases. In the lower panel, for a fixed weight graph node number $N = 100$ and weight graph edge density $D = 0.8$, the performance of the compiler is depicted as a function of the QAOA circuit depth p for a fixed weight graph with $N = 100$ nodes and edge density $D = 0.8$. The circuit depth and the number of CNOT gates compiled by Qcover are dozens of times smaller than those compiled by Qiskit, and this performance improvement remains almost unaffected by the QAOA depth.

3.5. Backends

As aforementioned, Quafu-Qcover incorporates a range of classical simulator backends and quantum hardware backends. The classical simulator backends include Pyquafu,[49] Qiskit,[51] ProjectQ,[52] Qulacs,[53] $t|ket\rangle$,[54] Quimb,[55] and Qton.[56] The specific details and instructions for using these simulators can be found in their respective software documentation, and will not be repeated here. This section aims to introduce the quantum hardware backends provided by the Quafu quantum computing cloud platform.

Table 1. Information of quantum chips.

Chip	Qubits	Average T_1 (μs)	Average T_2 (μs)	CZ fidelity
ScQ-P10	10	30.878	3.877	95.9%
ScQ-P18	18	35.492	3.285	95.7%
ScQ-P136	136	8.585	10.471	92.4%

The Quafu cloud platform currently offers three superconducting quantum chips as its quantum computing backend. Specifically, the platform provides two one-dimensional architecture chips, the 10-qubit chip (ScQ-P10) and the 18-qubit chip (ScQ-P18). In addition, there is a two-dimensional architecture chip known as the ScQ-P136. In Table 1, we summarize the information on the average energy relaxation time (T_1), average phase coherence time (T_2), and average fidelity of the two-qubit CZ gate of these chips. It should be noted that superconducting quantum chips need regular recalibration due to various environmental factors, causing the parameter information in the table to be slightly different from the data given by the Quafu cloud platform. In future versions of Quafu-Qcover, we aim to improve the quantum backend by including a wider range of systems, such as ion trap and Rydberg atom systems, to provide a more comprehensive quantum computing platform.

4. Example demonstration

In this section, we will outline a comprehensive workflow that demonstrates how Quafu-Qcover is used for solving combinatorial optimization problems.

In the first step, it is necessary to represent the combinatorial optimization problem as a corresponding weight graph. In the *Qcover.applications* module, we provide two methods for constructing the weight graph. One available method is to create a weight graph from the QUBO matrix. While this approach is more general, it necessitates manual conversion of the combinatorial optimization problem into a QUBO model by the user. The method is implemented in the *common* class. Furthermore, we propose another method to create the weight graph from the original problem. Specifically, corresponding classes have been developed for various types of combinatorial optimization problems, allowing us to transform the original problem into a QUBO model and obtain the weight graph. The Quafu-Qcover currently includes common optimization problems such as Maxcut, graph coloring, number partitioning, set packing, and quadratic assignment. Please consult the Quafu-Qcover project documentation for additional information. In future versions of Qcover, we intend to incorporate additional types of combinatorial optimization problems. The followings are some example code snippets to obtain the weight graph from either the QUBO matrix or the original problem.

```
from Qcover.applications import common, max_cut,
    graph_color, number_partition

# Graph data are described by the networkx python package
# Obtain the weight graph from the QUBO matrix
ising_mat = common.get_ising_matrix(qubo_mat)
weight_graph = common.get_weights_graph(ising_mat)

# Obtain the weight graph from the Maxcut problem
nodes = [0, 1, 2, 3, 4, 5]
edges = [(0,1),(1,2),(2,3),(3,4),(4,5),(0,4),(1,3)]
problem_graph = nx.Graph()
problem_graph.add_nodes_from(nodes)
problem_graph.add_edges_from(edges)
mc = max_cut.MaxCut(graph=problem_graph)
weight_graph, _ = mc.run()

# Obtain the weight graph from the graph coloring problem
gc = graph_color.GraphColoring(graph=problem_graph,
    color_num=3)
weight_graph, _ = gc.run()

# Obtain the weight graph from the number partition problem
nlist = [13, 7, 5, 2, 34, 21, 9, 45]
np = number_partition.NumberPartition(number_list=nlist)
weight_graph, _ = np.run()
```

The second step involves using the *Qcover.core* module to determine the optimal parameters for the QAOA circuit. This module provides various classical simulation backends and optimizers. In future versions of Qcover, an iterative approach using quantum hardware will be implemented to directly identify the optimal parameters. The following example code demonstrates the calculation of optimal parameters for the QAOA circuit.

```
from Qcover.core import Qcover
from Qcover.backends import CircuitByQulacs
from Qcover.optimizers import COBYLA

# Set QAOA depth
p = 1

# Select a simulation backend
bc = CircuitByQulacs()

# Select an optimizer
optc = COBYLA(options={'tol': 1e-3, 'disp': True})

# Instantiate the Qcover object
qc = Qcover(weight_graph, p=p, optimizer=optc,
    backend=bc)

# Run Qcover to find the optimal parameters
res = qc.run()
optimal_parameters = res['Optimal parameter value']
```

The third step involves compiling the QAOA circuit into a physical quantum circuit for execution on the Quafu quantum cloud platform, using the optimal parameters and weight graph. After compiling the quantum circuit, it is sent to the cloud platform, which then assigns a unique task ID number. The status of the task can be monitored using this ID number. Once the task is completed, the circuit sampling results can be obtained. The Qcover compilation package provides preprocessing and visualization methods for the sampling results. At present, binary grouping problems such as Maxcut and number partitioning problems can be visualized, and there is also support for visualizing graph coloring problems. The following code example demonstrates this step.

```
from Qcover.compiler import CompilerForQAOA

# Set the API Token, select the quantum backend of the quafu
    cloud platform, and set the number of sampling
token = "Your API Token"
cloud_bc = 'ScQ-P18'
shots = 100

# Instantiate the Qcover compiler object
qcover_compiler = CompilerForQAOA(weight_graph, p=p,
    optimal_params=optimal_parameters, apitoken=token,
    cloud_backend=cloud_bc)

# Compile the weight graph into the Quafu quantum circuit
    supported by the selected cloud backend and send the
    quantum circuit to the Quafu cloud platform to return the
    task ID number.
task_id = qcover_compiler.send(wait=False, shots=shots,
    task_name='MaxCut')

# If you choose wait=Ture, you have to wait for the result to
    return. If you choose wait=False, you can execute the
    following command to query the result status at any time,
    and the result will be returned when the task is completed.
quafu_solver = qcover_compiler.task_status_query(task_id)
if quafu_solver:
    counts_energy =
        qcover_compiler.results_processing(quafu_solver)
    qcover_compiler.visualization(counts_energy,
        problem='MaxCut', solutions=2,
        problem_graph=problem_graph)
```

In Fig. 5(a), we exhibit a specific case of the Maxcut problem, which was subsequently submitted to the Quafu quantum cloud computing platform for computation as described earlier. The sampling results obtained from the quantum processor are shown in Fig. 5(d), with the optimal solutions highlighted in green. These optimal solutions are also visualized in Figs. 5(b) and 5(c).

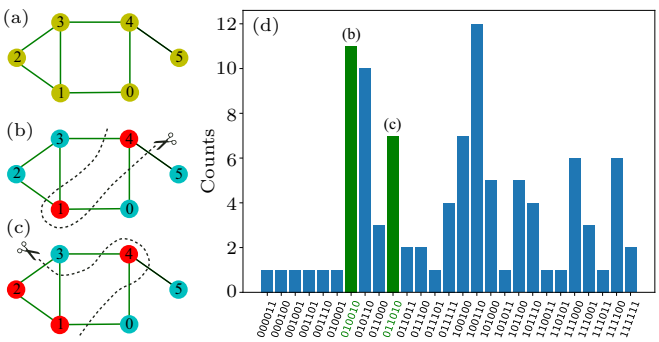

Fig. 5. (a) An example of the Maxcut problem. The nodes and edges are unweighted, or alternatively, it can be considered that the weights of the nodes and edges are all 1. (b) and (c) are two solutions given by the Quafu quantum computers. (d) Sampling results. The green highlights correspond to the two optimal solutions identified for (b) and (c).

5. Conclusion

At present, the Qcover does not support the direct searching of optimal parameters with quantum hardware due to the limitation of quantum resources. The quantum chips are only called for sampling tasks in the last stage. As part of our current efforts, the developing of a runtime module is underway to enable the submission of variational quantum circuit tasks, streamlining the iterative search for optimal parameters of the QAOA circuit on a quantum computer.

In conclusion, we have introduced Quafu-Qcover, an open-source cloud-based quantum software that allows users to remotely access the Quafu quantum computing cloud platform and use quantum computers to explore combinatorial optimization problems. Quafu-Qcover uses a graph decomposition algorithm to efficiently determine the optimal parameters for QAOA circuits. Furthermore, it provides a tailored high-performance QAOA circuit compiler that chooses the most reliable qubits for carrying out tasks according to the fidelity information of the quantum chip at the backend of Quafu. The workflow of Quafu-Qcover is intentionally designed to be straightforward and user-friendly. Users only need to input the corresponding QUBO model or weight graph of the combinatorial optimization problem and then utilize classical optimizers to find the optimal parameters for the QAOA circuit. Subsequently, after compilation by the compiler, the executable circuit is asynchronously sent to the Quafu quantum computing cloud platform.

Acknowledgements

Haifeng Yu, Meng-Jun Hu and Wei-Feng Zhuang are supported by the National Natural Science Foundation of China (Grant No. 92365206). Hong-Ze Xu acknowledges the support of the China Postdoctoral Science Foundation (Certificate Number: 2023M740272). Zheng-An Wang is supported by the National Natural Science Foundation of China (Grant No. 12247168) and China Postdoctoral Science Foundation(Certificate Number: 2022TQ0036).

References

[1] Childs A M and van Dam W 2010 *Rev. Mod. Phys.* **82** 1
[2] Montanaro A 2016 *npj Quantum Information* **2** 1
[3] Shor P 1994 *Algorithms for quantum computation: discrete logarithms and factoring Proceedings* 35*th Annual Symposium on Foundations of Computer Science* pp. 124–134
[4] Shor P W 1999 *SIAM Review* **41** 303
[5] Grover L K 1997 *Phys. Rev. Lett.* **79** 325
[6] Harrow A W, Hassidim A and Lloyd S 2009 *Phys. Rev. Lett.* **103** 150502
[7] Gulde S, Riebe M, Lancaster G P T, Becher C, Eschner J, Häffner H, Schmidt-Kaler F, Chuang I L and Blatt R 2003 *Nature* **421** 48
[8] Benhelm J, Kirchmair G, Roos C F and Blatt R 2008 *Nat. Phys.* **4** 463
[9] Taylor J M, Engel H A, Dür W, Yacoby A, Marcus C M, Zoller P and Lukin M D 2005 *Nat. Phys.* **1** 177
[10] Sau J D, Lutchyn R M, Tewari S and Das Sarma S 2010 *Phys. Rev. Lett.* **104** 040502
[11] Clarke J and Wilhelm F K 2008 *Nature* **453** 1031
[12] Barends R, Shabani A, Lamata L, *et al.* 2016 *Nature* **534** 222
[13] Arute F, Arya K, Babbush R, *et al.* 2019 *Nature* **574** 505
[14] Wu Y, Bao W S, Cao S, *et al.* 2021 *Phys. Rev. Lett.* **127** 180501
[15] O'Brien J L 2007 *Science* **318** 1567
[16] Wang X L, Chen L K, Li W, Huang H L, Liu C, Chen C, Luo Y H, Su Z E, Wu D, Li Z D, Lu H, Hu Y, Jiang X, Peng C Z, Li L, Liu N L, Chen Y A, Lu C Y and Pan J W 2016 *Phys. Rev. Lett.* **117** 210502
[17] Wang H, Qin J, Ding X, Chen M C, Chen S, You X, He Y M, Jiang X, You L, Wang Z, Schneider C, Renema J J, Höfling S, Lu C Y and Pan J W 2019 *Phys. Rev. Lett.* **123** 250503
[18] Zhong H S, Wang H, Deng Y H, Chen M C, Peng L C, Luo Y H, Qin J, Wu D, Ding X, Hu Y, Hu P, Yang X Y, Zhang W J, Li H, Li Y, Jiang X, Gan L, Yang G, You L, Wang Z, Li L, Liu N L, Lu C Y and Pan J W 2020 *Science* **370** 1460
[19] Kane B E 1998 *Nature* **393** 133
[20] Ladd T D, Goldman J R, Yamaguchi F, Yamamoto Y, Abe E and Itoh K M 2002 *Phys. Rev. Lett.* **89** 017901
[21] He Y, Gorman S K, Keith D, Kranz L, Keizer J G and Simmons M Y 2019 *Nature* **571** 371
[22] Preskill J 2018 *Quantum* **2** 79
[23] Bharti K, Cervera-Lierta A, Kyaw T H, Haug T, Alperin-Lea S, Anand A, Degroote M, Heimonen H, Kottmann J S, Menke T, Mok W K, Sim S, Kwek L C and Aspuru-Guzik A 2022 *Rev. Mod. Phys.* **94** 015004
[24] Farhi E, Goldstone J and Gutmann S 2014 arXiv: 1411.4028
[25] Bravyi S, Kliesch A, Koenig R and Tang E 2020 *Phys. Rev. Lett.* **125** 260505
[26] Egger D J, Mareček J and Woerner S 2021 *Quantum* **5** 479
[27] Patel Y J, Jerbi S, Bäck, T and Dunjko V 2024 *EPJ Quantum Technol.* **11** 6
[28] Farhi E, Gamarnik D and Gutmann S 2020 arXiv: 2004.09002
[29] McClean J R, Harrigan M P, Mohseni M, Rubin N C, Jiang Z, Boixo S, Smelyanskiy V N, Babbush R and Neven H 2021 *PRX Quantum* **2** 030312
[30] Akshay V, Philathong H, Morales M E S and Biamonte J D 2020 *Phys. Rev. Lett.* **124** 090504
[31] Stilck França D and García-Patrón R 2021 *Nat. Phys.* **17** 1221
[32] Dam W v, Eldefrawy K, Genise N and Parham N 2021 2021 *IEEE International Conference on Quantum Computing and Engineering (QCE)* pp. 160–170
[33] Wurtz J and Lykov D 2021 *Phys. Rev. A* **104** 052419
[34] Alsina D and Latorre J I 2016 *Phys. Rev. A* **94** 012314
[35] Wang Y, Li Y, Yin Z q and Zeng B 2018 *npj Quantum Information* **4** 46
[36] Harper R and Flammia S T 2019 *Phys. Rev. Lett.* **122** 080504
[37] Huang W J, Chien W C, Cho C H, Huang C C, Huang T W and Chang C R 2020 *Quantum Engineering* **2** e45
[38] Ball P 2021 *Nature* **599** 542
[39] Quafu quantum computing cloud platform https://quafu.baqis.ac.cn,
[40] Zhuang W F, Pu Y N, Xu H Z, Chai X, Gu Y, Ma Y, Qamar S, Qian C, Qian P, Xiao X, Hu M J and Liu D E 2021 arXiv: 2112.11151
[41] Botea A, Kishimoto A and Marinescu R 2018 *On the complexity of quantum circuit compilation Proceedings of the International Symposium on Combinatorial Search* vol. 9 pp. 138–142
[42] Siraichi M Y, Santos V F d, Collange C and Pereira F M Q 2018 *Qubit allocation Proceedings of the* 2018 *International Symposium on Code Generation and Optimization* CGO 2018 (New York, NY, USA: Association for Computing Machinery) pp. 113–125
[43] Wille R and Burgholzer L 2023 *Proceedings of the* 2023 *International Symposium on Physical Design* ISPD '23 (New York: Association for Computing Machinery) pp. 198–204
[44] Lao L and Browne D E 2022 *Proceedings of the* 49*th Annual International Symposium on Computer Architecture* ISCA '22 (New York: Association for Computing Machinery) pp. 351–365
[45] Alam M, Saki A A and Ghosh S 2020 57*th ACM/IEEE Design Automation Conference (DAC)* pp. 1–6

[46] Alam M, Ash-Saki A and Ghosh S 2020 53*rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)* pp. 215–228
[47] Weidenfeller J, Valor L C, Gacon J, Tornow C, Bello L, Woerner S and Egger D J 2022 *Quantum* **6** 870
[48] Jin Y, Fong L, Chen Y, Hayes A B, Zhang S, Zhang C, Hua F, Z E and Zhang 2021 arXiv: 2112.06143
[49] Glover F, Kochenberger G, Hennig R and Du Y 2022 *Annals of Operations Research* **314** 141
[50] Neven H, Denchev V S, Drew-Brook M, Zhang J, Macready W G and Rose G 2009 *Quantum* **4** 1
[51] Qiskit contributors 2023 *Qiskit: An open-source framework for quantum computing* https://github.com/Qiskit
[52] Steiger D S, Häner T and Troyer M 2018 *Quantum* **2** 49
[53] Suzuki Y, Kawase Y, Masumura Y, Hiraga Y, Nakadai M, Chen J, Nakanishi K M, Mitarai K, Imai R, Tamiya S, Yamamoto T, Yan T, Kawakubo T, Nakagawa Y O, Ibe Y, Zhang Y, Yamashita H, Yoshimura H, Hayashi A and Fujii K 2021 *Quantum* **5** 559
[54] Sivarajah S, Dilkes S, Cowtan A, Simmons W, Edgington A and Duncan R 2020 *Quantum Science and Technology* **6** 014003
[55] Gray J 2018 *Journal of Open Source Software* **3** 819
[56] Qton https://github.com/thewateriswide/qton 2.1
[57] Zhou L, Wang S T, Choi S, Pichler H and Lukin M D 2020 *Phys. Rev. X* **10** 021067
[58] Lloyd S 1996 *Science* **273** 1073
[59] Suzuki M 1991 *Journal of Mathematical Physics* **32** 400